Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Concepto prueba unitaria
- Introducción a JUNIT
- Anotaciones (After, Test, Before)
- Principales Asserts en JUNIT
- Creación y Ejecución de una prueba unitaria simple con Junit en NetBeans.

idat

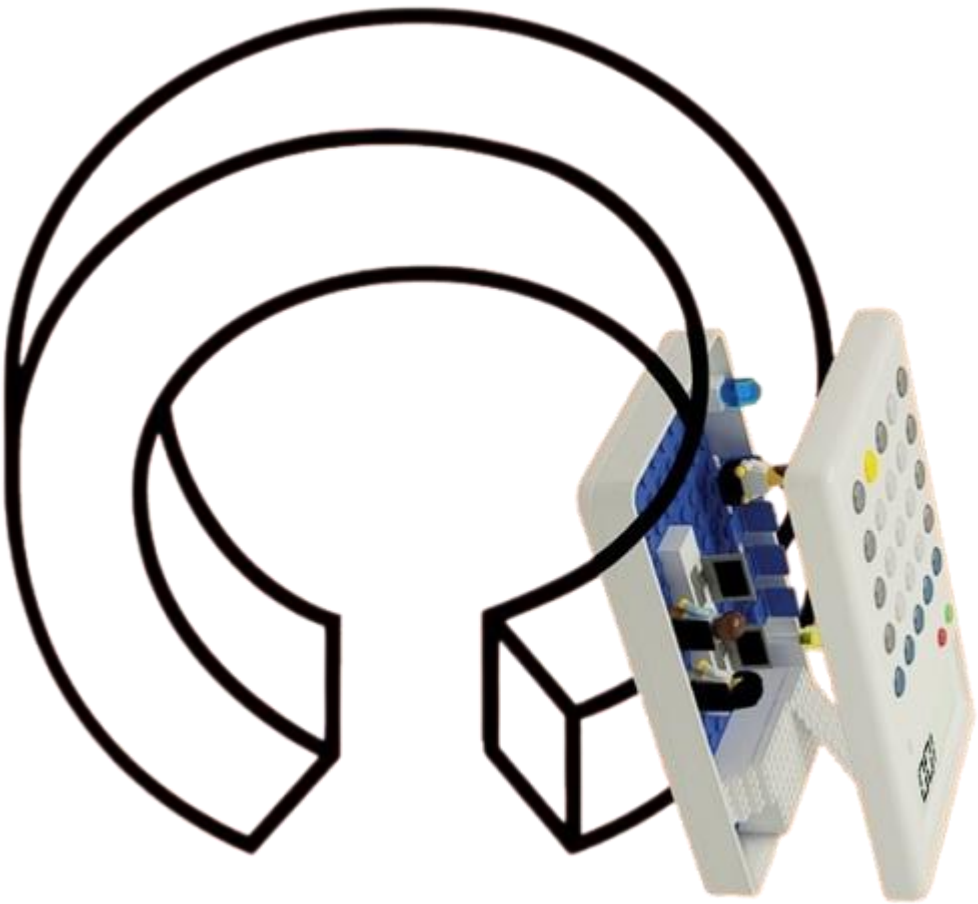
Conversemos

http://www.menti.com

idat

Introducción al Selenium WebDriver – Parte 1

Semana 9



idat

Logro de aprendizaje

- Conceptos
- Arquitectura de Selenium
- Tipos de selenium
- Drivers selenium según explorador
- Localizadores
- Principales comandos
- Configuración del entorno de pruebas.

Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- Realice la configuración del entorno de pruebas usando gradle, java y selenium web driver según lo revisado en clase.

idat

Logro de aprendizaje

Realiza la configuración del entorno de trabajo del IDE Selenium Web Driver para automatización de pruebas funcionales.

idat

Test cases & User stories

Test case:

- Es un grupo de condiciones bajo las cuales se puede determinar si un requerimiento ha sido o no cumplido.

User story:

- Es una o mas oraciones en el lenguaje de negocios del usuario que captura lo que el usuario hace o necesita hacer como parte de sus tareas funcionales. Captura el “quien”, el “que” y el “como” de un requerimiento.

idat

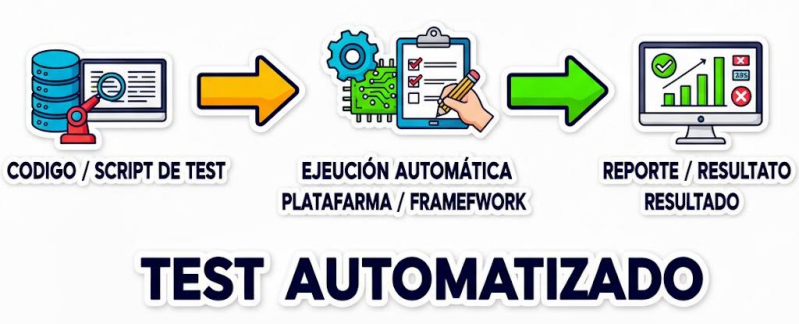
Testing Manual y Automatizado

Tests Manuales:

- Tests que son ejecutados por usuarios, los cuales pueden ir recorriendo la aplicación paso a paso en busca de defectos.

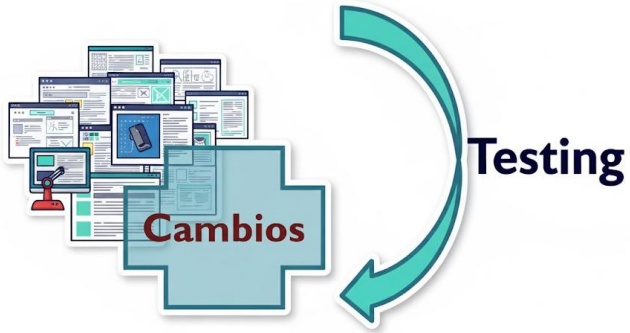
Tests Automatizados:

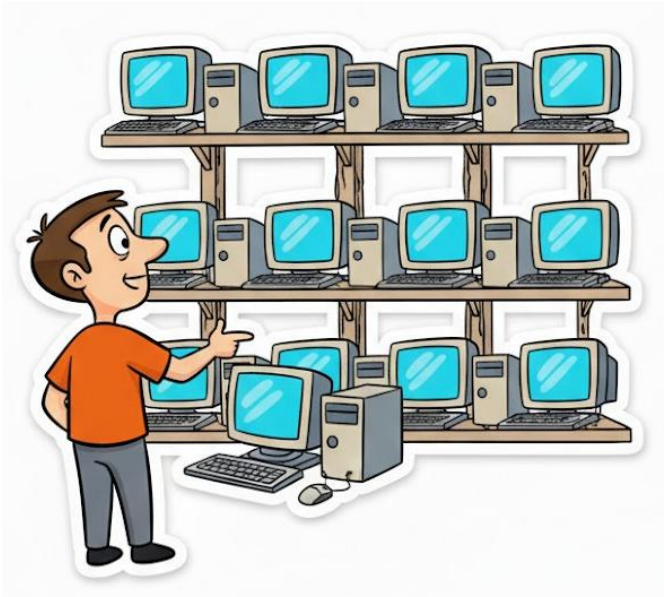
- Tests que son ejecutados por herramientas específicas, las cuales requieren una mínima supervisión.



idat

Automation Testing





idat

Ventajas del Test automatizado

MANUAL	AUTOMATIZADO
El éxito depende de la pericia de quién la ejecuta.	El resultado no depende de quién la ejecuta.
Difícil de documentar	Autodocumentada
Difícil de mantener actualizado	Auto actualizable ante cambios en el aplicativo
Mas lento y tedioso	Mas rápido
	Repetible
	Reutilizable

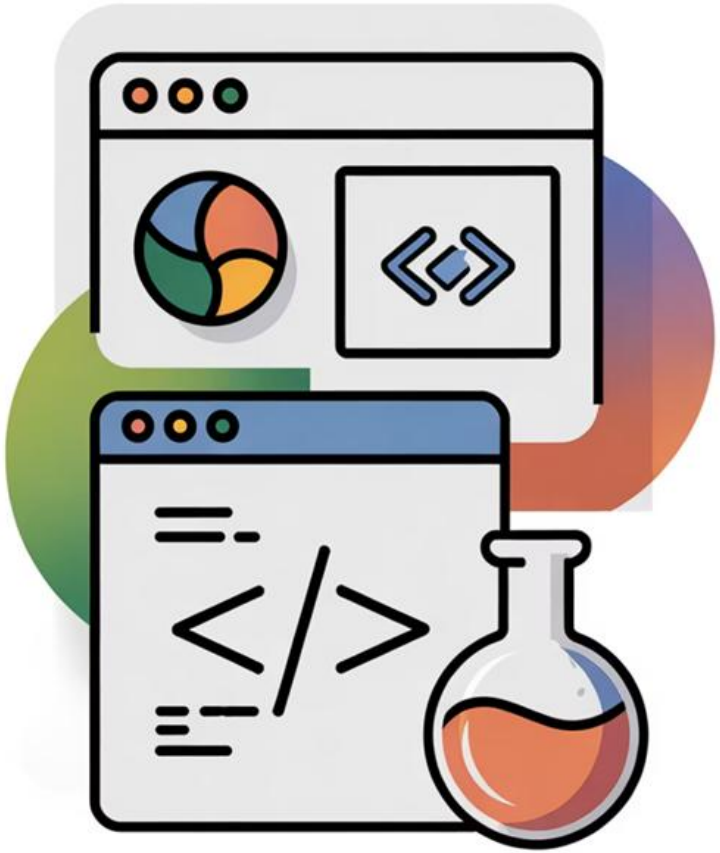
idat

¿Cuándo automatizar?

- ✔ Tests que deban ser ejecutados cada vez que la aplicación sea modificada.
- ✔ Tests Smoke y de regresión.
- ✔ Test a ser ejecutados en muchos ambientes.
- ✘ Test de Usabilidad
- ✘ Tests que se ejecutan una única vez
- ✘ Aplicaciones inestables.

idat

¿Qué es Selenium?



Suite integral de herramientas diseñada para automatizar pruebas en pruebas en aplicaciones web, permitiendo validación eficiente y y repetible.

Múltiples Lenguajes

Java, Python, C#, JavaScript, Ruby

Compatibilidad Universal Universal

Navegadores y sistemas operativos diversos

Código Abierto

Comunidad activa y soporte continuo

idat

¿Qué es Selenium?

Selenium WebDriver es una herramienta de automatización de pruebas funcionales para aplicaciones web que permite controlar navegadores de forma programática, simulando la interacción real de un usuario.

Permite:

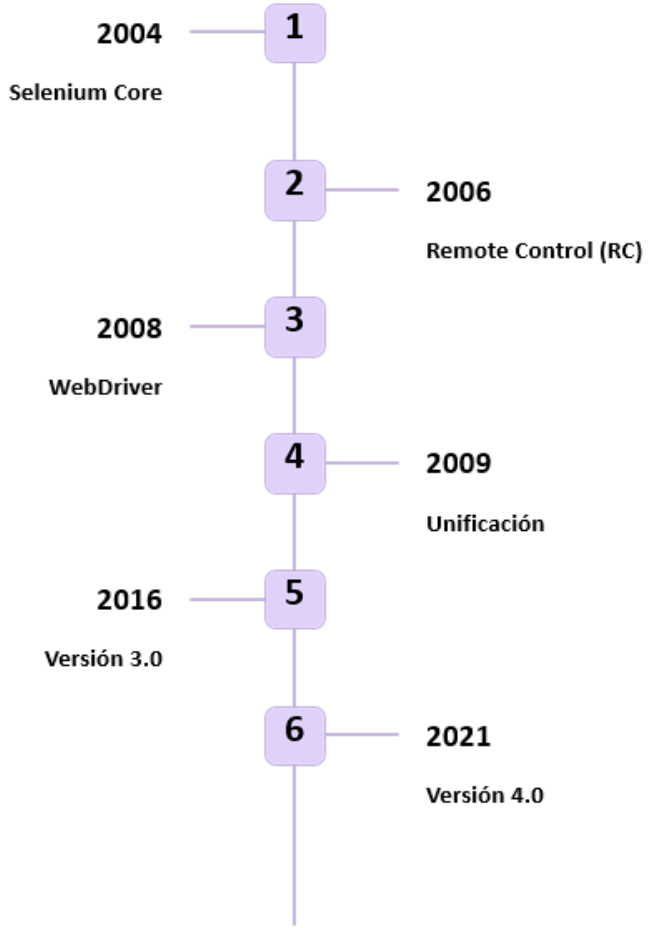
- Abrir navegadores
- Hacer clic en botones
- Escribir texto
- Navegar entre páginas
- Validar resultados
- Ejecutar pruebas repetitivas automáticamente

Es la tecnología principal del ecosistema Selenium y reemplaza al antiguo Selenium RC.

idat

La Evolución de Selenium

Desde su creación en 2004, Selenium ha evolucionado transformando la automatización web.



idat

Automatización de Pruebas

Ejecuta casos de prueba automáticamente, reduciendo tiempo y esfuerzo mientras incrementas cobertura incrementas cobertura y precisión.

1 **Ejecución Automática**

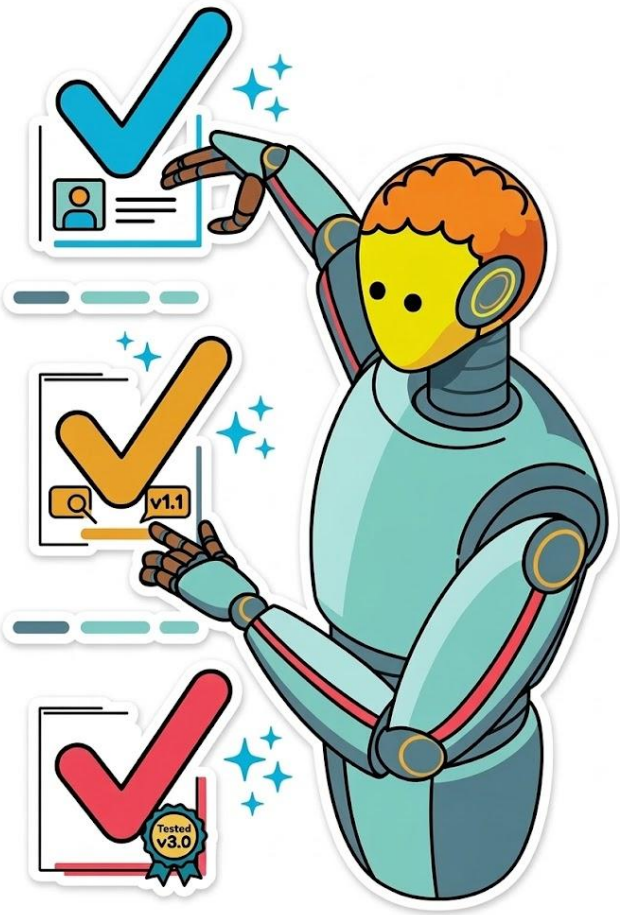
Casos de prueba se ejecutan sin intervención humana, permitiendo validaciones repetitivas y consistentes.

2 **Eficiencia Mejorada**

Reducción significativa de tiempo y esfuerzo en ciclos de prueba, liberando recursos para tareas estratégicas.

3 **Mayor Cobertura**

Alcanza escenarios complejos y casos extremos con precisión superior a pruebas manuales.



idat

Pruebas Web: Alcance Completo

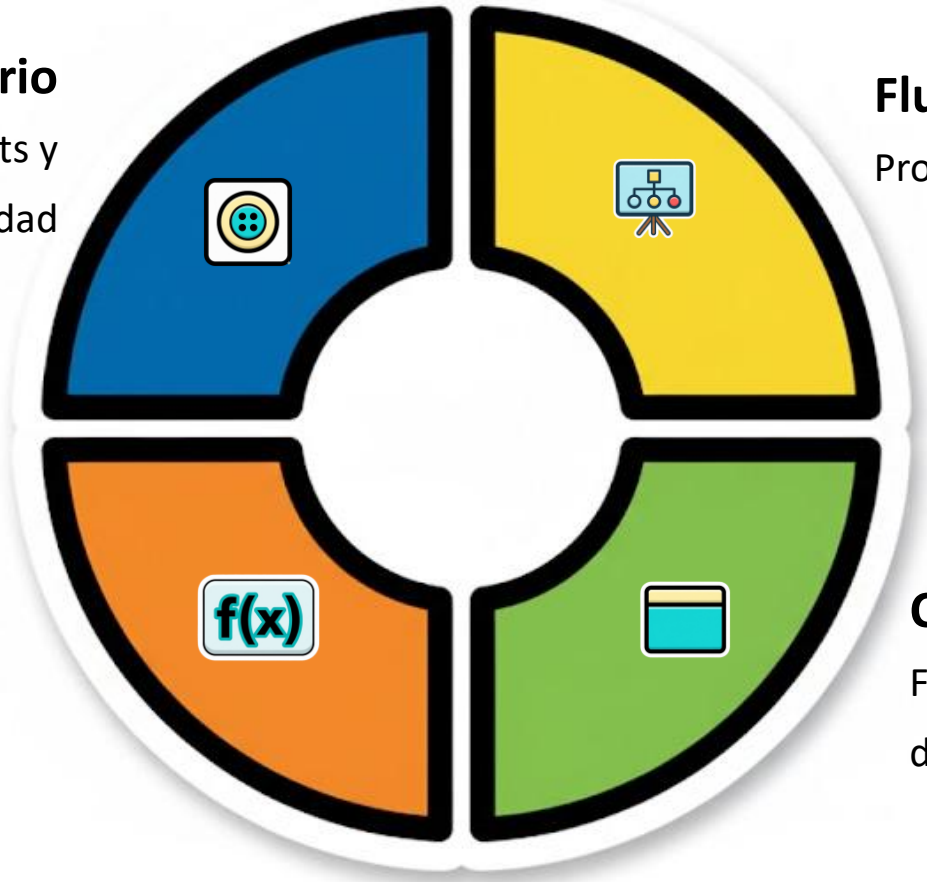
Valida cada aspecto de tu aplicación web, desde la interfaz hasta los flujos de negocio críticos.

Interfaz de Usuario

Elementos visuales, layouts y responsividad

Flujos de Negocio

Procesos completos y transacciones críticas



Funcionalidades

Validación de características y comportamientos

Compatibilidad

Funcionamiento uniforme en navegadores diversos

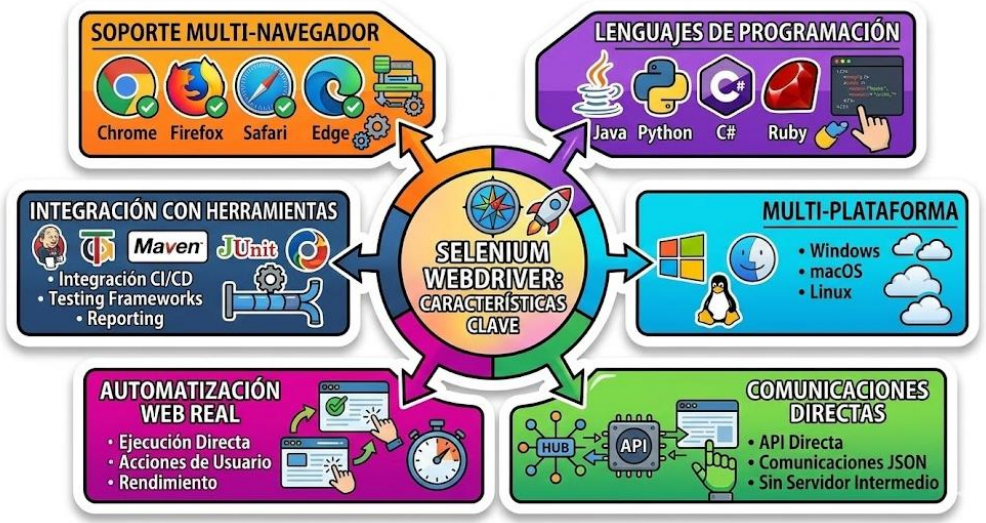
idat

WebDriver: El Corazón de Selenium

Interfaz potente que comunica directamente con navegadores, simulando acciones reales de usuarios con usuarios con precisión y confiabilidad.

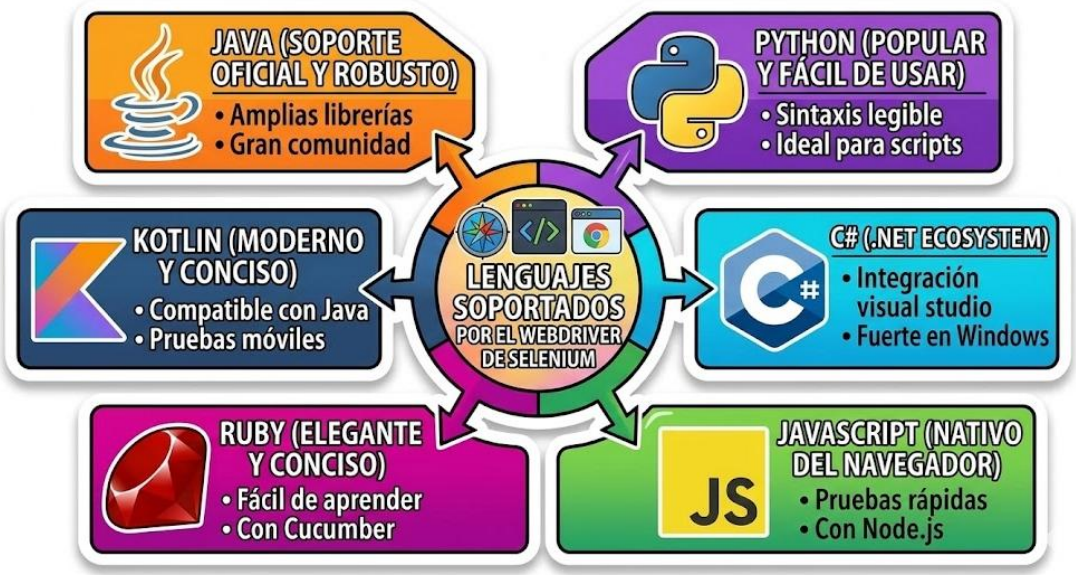
Características Clave

- Comunicación directa con navegadores
- API intuitiva y flexible
- Soporte multiplataforma
- Ejecución rápida y confiable



Lenguajes Soportados

Java, Python, C#, JavaScript, Ruby, PHP y otros

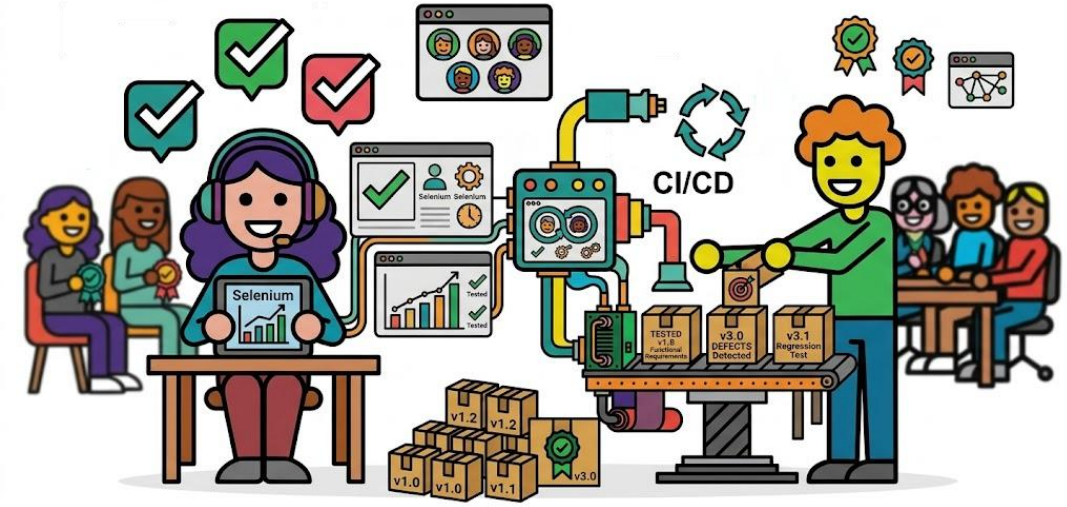


idat

Objetivo de Selenium

El objetivo principal de Selenium WebDriver es:
Automatizar pruebas funcionales de aplicaciones web para:

- Verificar que el sistema cumpla los requisitos
- Reducir pruebas manuales repetitivas
- Detectar defectos tempranamente
- Ejecutar pruebas de regresión rápidamente
- Integrarse con Integración Continua (CI/CD)
- Mejorar calidad y cobertura de pruebas



idat

Elementos Web y Localizadores

Identifica con precisión cada componente en tu página web utilizando estrategias de localización robustas y confiables.

Elementos Base
Botones, campos de texto, enlaces, formularios, tablas y menús interactivos

Estrategias de Búsqueda
ID, Name, Class Name, Tag Name, CSS Selector, XPath, Link Text

Selectores Avanzados
Combinaciones XPath y CSS para elementos dinámicos y complejos

idat

Estrategias de localización para elementos

Una página web es un documento en la World Wide Web que se puede visualizar en un navegador. Una página web contiene bastante contenido, como título, párrafos, imágenes, listas, enlaces y más.

Los usuarios interactúan visualmente con las páginas web, un aspecto crucial. Pueden desplazarse para ver más texto en imágenes grandes e incluso hacer clic en enlaces para acceder a nuevos artículos. No necesitan conocer comandos específicos, y los sitios web que ofrecen una buena experiencia de usuario hacen que las interacciones sean intuitivas y sencillas.

idat

Estrategias de localización para elementos

¿Qué hace que una página web sea interactiva?

1. **HTML (Lenguaje de Marcado de Hipertexto)**
   Funciona como el esqueleto de la página web. Establece las bases para todo el contenido. HTML es un lenguaje de marcado, no un lenguaje de programación propiamente dicho. Simplemente indica qué se debe mostrar.

```
<!-- Formato HTML -->
<tagname attribute="value"> textContent </tagname>
```

idat

Estrategias de localización para elementos

¿Qué hace que una página web sea interactiva?

2. **CSS (Hojas de estilo en cascada)**
   Funciona como la interfaz de la página web, donde se pueden añadir estilos y controlar aspectos como el color, el tamaño, la fuente, la posición y el diseño. Se puede usar una etiqueta HTML con `style` o un archivo `.css` independiente para mayor reutilización.

```
body {
  background-color: lightblue;
}

.main-content {
  Font-family: Helvetica;
}
```

idat

Estrategias de localización para elementos

¿Qué hace que una página web sea interactiva?

3. **JavaScript**

   El contenido dinámico se genera mediante JavaScript, el motor de la página. Dado que los principales navegadores web son compatibles con JavaScript, prácticamente todas las aplicaciones en línea actuales lo utilizan para ofrecer una experiencia de usuario dinámica en la interfaz. El código JavaScript puede escribirse directamente en archivos HTML o, como es recomendable, en .js archivos separados.

```
alert("¡Aquí hay algo de JavaScript!");
```

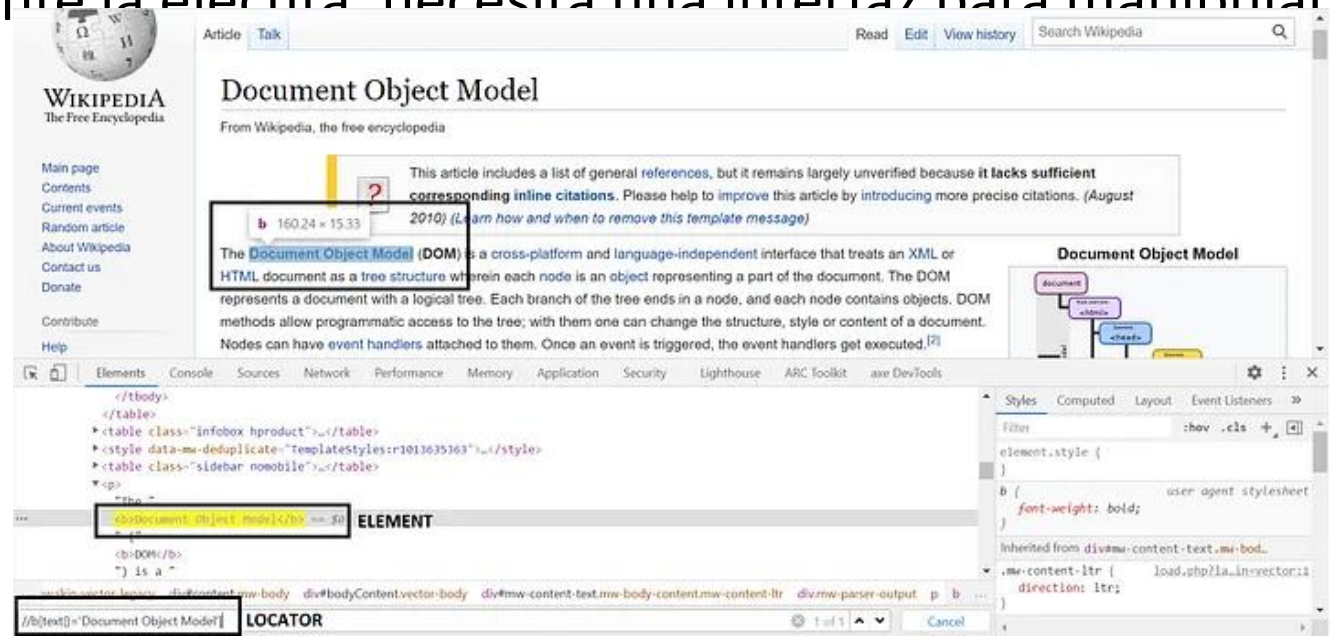
idat

Estrategias de localización para elementos

Programación con DOM

El Modelo de Objetos del Documento (DOM) es una interfaz de programación para documentos HTML y XML. HTML, CSS y JavaScript son documentos codificados que un navegador web combina para crear una página web visual. Cuando un navegador renderiza una página y posteriormente la ejecuta, necesita una interfaz para manipularla.

Obtener los elementos en sí es el primer paso en la programación DOM. Esto subraya un punto clave: existe una distinción entre un elemento y su localizador.



idat

Estrategias de Localización soportadas por Selenium

Se implementan con:

Java

```
driver.findElement(By.id("usuario"));
```

Python

```
driver.find_element(By.ID, "usuario")
```

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

1. **ID:** Localiza el elemento por el atributo **id** del HTML.
   Sintaxis: *By.id("loginBtn")*
   Ejemplo HTML: *<input id="usuario" type="text">*
   Uso: *driver.findElement(By.id("usuario")).sendKeys("admin");*

✅ Ventajas
- ✓ Más rápido
- ✓ Único (debería ser único por norma HTML)
- ✓ Más estable
- ✓ Fácil lectura

❌ Limitaciones
- ✕ Algunos sistemas generan IDs dinámicos

🎯 **Recomendación**

👉 **Primera opción siempre que exista**

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

2. **Name:** Localiza por el atributo **name**.
   Sintaxis: *By.name("email")*
   Ejemplo HTML: *<input name="email">*

✅ Ventajas
- ✓ Sencillo
- ✓ Común en formularios

❌ Limitaciones
- ✕ Puede no ser único

🎯 **Recomendación**

👉 Útil en formularios simples

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

3. **ClassName:** Localiza por el atributo **class**.
   Sintaxis: *By.className("btn-primary")*
   Ejemplo HTML: *<button class="btn-primary">Guardar</button>*

✅ Ventajas
✓ Fácil identificación visual

❌ Limitaciones
✕ Muchas clases no son únicas
✕ Cambian con frecuencia (frameworks CSS)

🎯 **Recomendación**

👉 Usar solo si la clase es única

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

4. **TagName:** Localiza por etiqueta HTML.
   Sintaxis: *By.tagName("input")*
   Ejemplo HTML: *<input>*

✅ Ventajas
✓ Útil para listas o conteos

❌ Limitaciones
✕ Muy genérico
✕ No identifica elementos únicos

🎯 **Uso típico**

```
driver.findElements(By.tagName("a"));
```

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

4. **LinkText:** Localiza enlaces por texto exacto visible.
   Sintaxis: *By.linkText("Iniciar sesión")*
   Ejemplo HTML: *<a>Iniciar sesión</a>*

✅ Ventajas
- ✓ Muy legible
- ✓ Ideal para enlaces de menú

❌ Limitaciones
- ✕ Sensible a cambios de texto
- ✕ Problemas con idiomas

🎯 **Recomendación**

👉 Bueno para navegación simple

idat

Estrategias de Localización soportadas por Selenium

Tipos de Localizadores

5. **XPath:** Lenguaje de consulta para navegar por el DOM.
   Sintaxis: *By.xpath("//input[@id='usuario']")*

📌 Tipos de XPath

Absoluto (NO recomendado)

```
//html/body/div[2]/form/input[1]
```

Relativo (recomendado)

```
//input[@id='usuario']
```

📌 Ejemplos útiles

XPath	Función
//input	todos los input
//button[@type='submit']	por atributo
//div[contains(text(),'Login')]	texto parcial
//input[@name='user' and @type='text']	múltiples condiciones

✅ Ventajas

- ✓ Muy potente
- ✓ Localiza elementos complejos
- ✓ Permite navegar jerarquías

❌ Limitaciones

- ✕ Más lento
- ✕ Difícil mantenimiento
- ✕ Frágil ante cambios de estructura

🎯 Recomendación

👉 Usar **solo cuando CSS no sea suficiente**

Framework de Pruebas: Estructura y Organización

Construye pruebas escalables y mantenibles con gestión centralizada de datos, configuraciones y reportes.

Estructura Base

Organización modular de código y componentes

Gestión de Datos

Archivos de configuración, datos de prueba y fixtures

Reportes

Informes ejecutivos con evidencia y análisis detallado

Configuraciones

Entornos, navegadores y parámetros centralizados

idat

Arquitectura de Selenium

- Código de Prueba

Scripts en tu lenguaje favorito

- WebDriver API

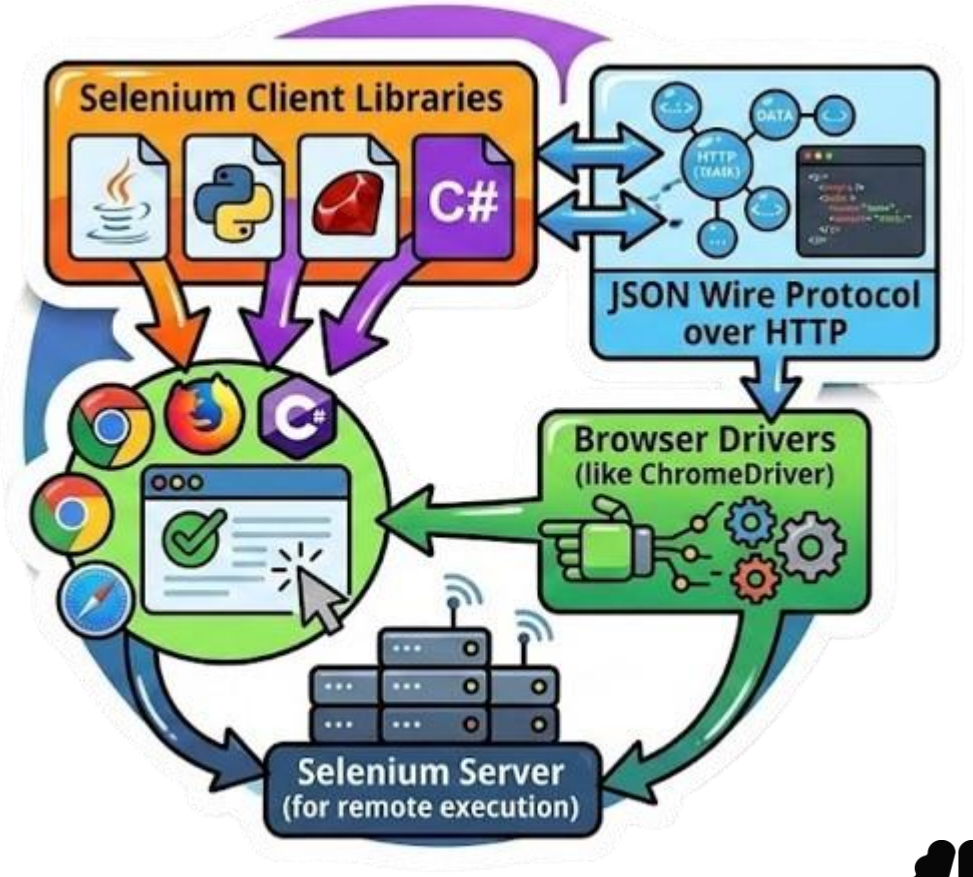
Interfaz de comunicación

- Browser Drivers

Controladores específicos

- Navegadores

Chrome, Firefox, Safari, Edge



idat

¿Dudas?

¿Miedos?

¿Temores?

idat

Laboratorio: JUnit

idat

Ejecutar Apache Netbeans

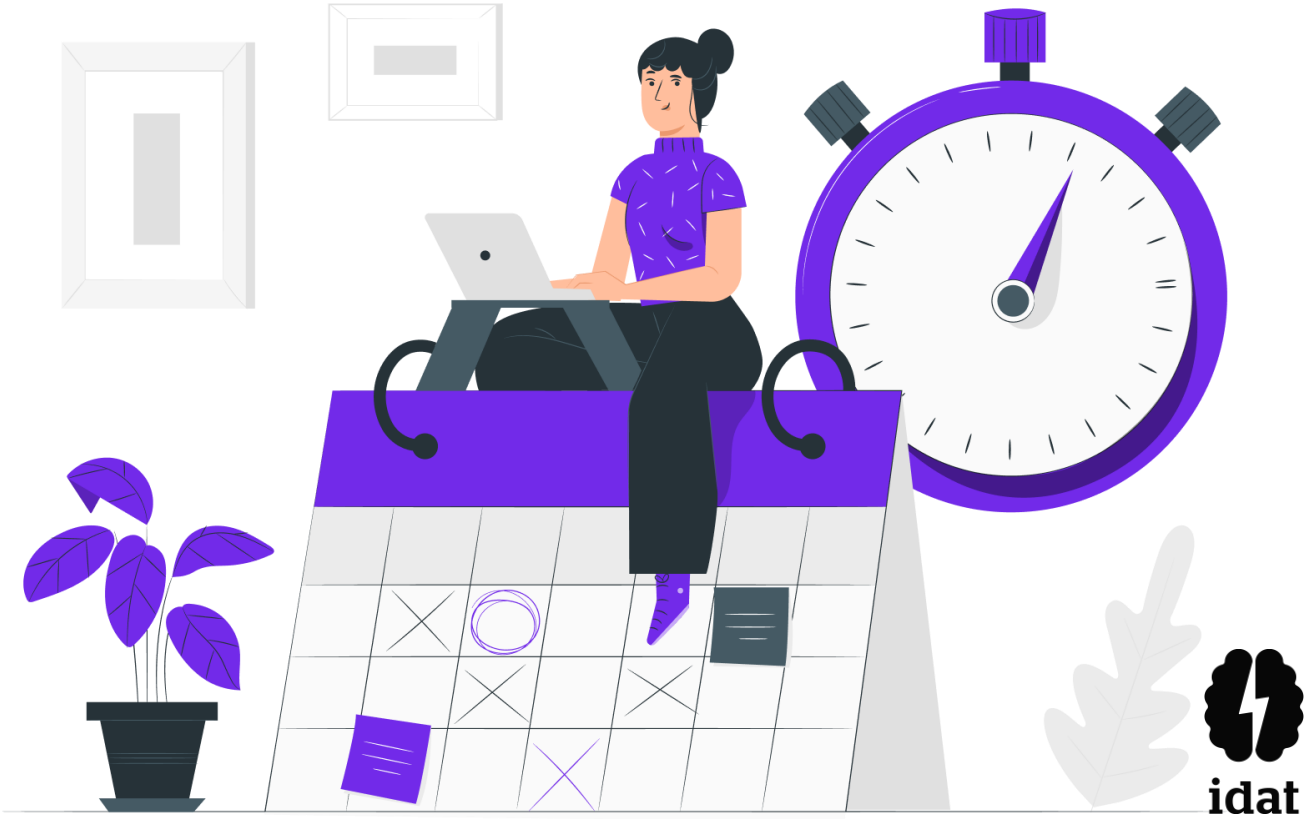
- Cree el Proyecto “ejemploJUnit-Inventario”
- Modifique el pom.xml
- Cree la Clase: inventario
  - atributo: entero nivelStock;
  - constructor: inicializa el stock inicial
  - métodos:
    - void agregarStock(int cantidad): Aumenta el stock en la cantidad enviada
    - boolean removerStock(int cantidad): Verdadero si disminuye, Falso si no se puede
    - int getNivelStock(): Devuelve el nivel de stock

- Cree la Clase: inventarioTest
  - Inicialice el stock inicial
  - Pruebe agregando stock
  - Pruebe remover stock

idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS